

No se ha podido establecer conexión con el servicio reCAPTCHA. Comprueba tu conexión a Internet y vuelve a cargar la página para ver otro reCAPTCHA.

Actividad Guiada Algoritmos de Optimización 1

Nombre: Efraím Alberto Casas Herrera

Enlace: <https://colab.research.google.com/drive/1Vbug5H9WS-AteUS7Q-lqTV8EytDrKcvW?usp=sharing>

<https://github.com/eacasas/algoritmosOptimizacion>

▼ Divide y vencerás

▼ Torres de Hanoi

```
#Torres de Hanoi

def Torres_Hanoi(N, desde, hasta):
    #N: número de fichas
    #desde: número de poste
    #hasta: número hasta

    if N==1:
        print(f'Lleva la ficha desde {desde}, hasta {hasta}')
    else:
        Torres_Hanoi(N-1, desde, 6-desde-hasta)
        print(f'Lleva la ficha desde {desde}, hasta {hasta}')
        Torres_Hanoi(N-1, 6-desde-hasta,hasta )

##Prueba
Torres_Hanoi(4,1,3)

Lleva la ficha desde 1, hasta 2
Lleva la ficha desde 1, hasta 3
Lleva la ficha desde 2, hasta 3
Lleva la ficha desde 1, hasta 2
Lleva la ficha desde 3, hasta 1
Lleva la ficha desde 3, hasta 2
Lleva la ficha desde 1, hasta 2
Lleva la ficha desde 1, hasta 3
Lleva la ficha desde 2, hasta 3
Lleva la ficha desde 2, hasta 1
Lleva la ficha desde 3, hasta 1
Lleva la ficha desde 2, hasta 3
Lleva la ficha desde 1, hasta 2
Lleva la ficha desde 1, hasta 3
Lleva la ficha desde 2, hasta 3
```

▼ Técnica Voráz

Se ha guardado correctamente

```
def cambio_monedas(cantidad, sistema):
    #sistema = sistema.sort(reverse=True)
    # cantidad a cambiar
    # sistema: valor de las monedas diferente, ordenado de mayor valor a menor
    solucion = {}
    for v in sistema:
        monedas = cantidad // v
        solucion[v] = monedas
        cantidad -= monedas * v
    if cantidad == 0:
        return solucion

cambio_monedas(180, [50,20,10,5,2])

{50: 3, 20: 1, 10: 1}
```

▼ Vuelta Atrás (Backtracking)

▼ El problema de las N reinas

```
#N Reinas - Vuelta Atrás()
#####

#Verifica que en la solución parcial no hay amenazas entre reinas
#####
def es_prometedora(SOLUCION,etapa):
#####
    #print(SOLUCION)
    #Si la solución tiene dos valores iguales no es valida => Dos reinas en la misma fila
    for i in range(etapa+1):
        #print("El valor " + str(SOLUCION[i]) + " está " + str(SOLUCION.count(SOLUCION[i])) + " veces")
        if SOLUCION.count(SOLUCION[i]) > 1:
            return False

    #Verifica las diagonales
    for j in range(i+1, etapa +1 ):
        #print("Comprobando diagonal de " + str(i) + " y " + str(j))
        if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
    return True

#Traduce la solución al tablero
#####
def escribe_solucion(S):
#####
    n = len(S)
    for x in range(n):
        print("")
        for i in range(n):
            if S[i] == x+1:
                print(" X " , end="")
            else:
                print(" - ", end="")

#Proceso principal de N-Reinas
#####
def reinas(N, solucion=[],etapa=0):
#####
    ## ...
    if len(solucion) == 0:          # [0,0,0...]
        solucion = [0 for i in range(N) ]

    for i in range(1, N+1):
        solucion[etapa] = i
        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)

    solucion[etapa] = 0

# Prueba
reinas(4,solucion=[],etapa=0)

[2, 4, 1, 3]
[3, 1, 4, 2]
```

Se ha guardado correctamente



None

▼ Práctica Individual

▼ Puntos cercanos

```
import numpy as np
import pandas as pd
import time
from numpy.random import default_rng

#

lista_1D = np.random.randint(0,100, size = 10)
display(lista_1D)

array([59,  9, 75, 63, 65, 72, 45, 84, 35, 86])
```

▼ 1. Solución fuerza bruta

En esta solución se realiza la comparación entre cada uno de los elementos de la lista.

```
def minDistanciaFB(lista_1D):
    distancias=[]
    for i in lista_1D:
        for j in lista_1D:
            if i!=j:
                distancias.append((abs(i-j),(i,j)))

    return (min(distancias, key=lambda x: x[0]))

minimo = minDistanciaFB(lista_1D)
print(f'los números mas cercanos son: {minimo[1]} con una distancia de: {minimo[0]}')

los números mas cercanos son: (63, 65) con una distancia de: 2
```

Oreden de complejidad

El orden de complejidad del algoritmo es $O(n^2)$

▼ 2. Divide y vencerás

Para la estrategia de divide y vencerás utilizaremos primero un ordenamiento QuickSort(divide y vencerás) y luego realizaremos la comprobación de distancia entre números adyacentes. Con esto se espera reducir el Orden de complejidad promemedio.

```
def partition(array, low, high):

    pivot = array[high]
    i = low - 1

    for j in range(low, high):
        if array[j] < pivot:
            i = i + 1
            (array[i], array[j]) = (array[j], array[i])

    # Swap the pivot element with the greater element specified by i
    (array[i + 1], array[high]) = (array[high], array[i + 1])
    # Return the position from where partition is done
    return i + 1

def quickSort(array, low, high):
    if low < high:
        pi = partition(array, low, high)
        # Recursive call on the left of pivot
        quickSort(array, low, pi - 1)
        # Recursive call on the right of pivot
        quickSort(array, pi + 1, high)
```

Se ha guardado correctamente

```
def minDistanciaDV(lista_1D):

    quickSort(lista_1D,0,len(lista_1D)-1)
    min = 100000000
    for i in range(len(lista_1D)-1):
        if abs(lista_1D[i]-lista_1D[i-1]) < min:
            min=abs(lista_1D[i]-lista_1D[i-1])
            tupla = (lista_1D[i],lista_1D[i-1])

    return min, tupla


minimoDV, tupla = minDistanciaDV(lista_1D)
print(lista_1D)
print(f'los números mas cercanos son: {tupla} con una distancia de: {minimoDV}')
```



```
[ 9 35 45 59 63 65 72 75 84 86]
los números mas cercanos son: (65, 63) con una distancia de: 2
```

Orden de complejidad

El orden de complejidad de la solución utilizando Divide y Vencerás, está dado por el Orden de complejidad del método de ordenamiento QuickSort mas el orden de complejidad de recorrer el arreglo:

$O(O(Qs)+O(n))$

El orden de complejidad de QuickSort en promedio es $O(n \log n)$ y el peor de los casos es $O(n^2)$. Entonces el orden de la función sería:

Promedio: $O(n \log n + n) \Rightarrow O(n \log n)$: Cuasi lineal

Peor de los casos: $O(n^2 + n) \Rightarrow O(n^2)$: Cuadrático (Esta caso es muy poco probable, para números aleatorios. porque el arreglo debe estar en orden inverso)

▼ Pruebas

```
rng = default_rng()
lista_1000 = rng.choice(4000, size=1000, replace=False)

display(lista_1000)
```

Se ha guardado correctamente



Se ha guardado correctamente

▼ Prueba masiva para 10 muestras de cada método

A continuación se realizará una prueba con 10 muestras por cada método, y se guardarán los resultados en la lista "resultados" para posteriormente visualizar los resultados mediante la libreria Seaborn.

```
#Para almacenar los resultados
resultados = pd.DataFrame(columns=['metodo','n','tiempo'])

#Para generar números aleatorios sin repetición
rng = default_rng()

#Se realizarán 10 muestras por cada método
for i in range(10):
    # Se genera la lista para ser probada con ambos métodos
    lista_1000 = rng.choice(4000, size=1000, replace=False)

    #Se llama el método de fuerza bruta: FB
    t0 = time.perf_counter()
    minimo = minDistanciaFB(lista_1000)
    r= time.perf_counter()-t0
    new_row = {'metodo':'FB','n':i,'tiempo':r}
    resultados.append(new_row, ignore_index=True)

    #Se llama el método de Divide y Vencerás
    t0 = time.perf_counter()
    minimoDV, tupla = minDistanciaDV(lista_1000)
    r= time.perf_counter()-t0
    new_row = {'metodo':'DV','n':i,'tiempo':r}
    resultados.append(new_row, ignore_index=True)
```

```
display(resultados)
```

	metodo	n	tiempo
0	FB	0	1.219759
1	DV	0	0.004643
2	FB	1	0.993516
3	DV	1	0.005402
4	FB	2	1.031462
5	DV	2	0.004835
6	FB	3	1.068465
7	DV	3	0.004820
8	FB	4	0.934809
9	DV	4	0.005300
10	FB	5	1.060120
11	DV	5	0.004720
12	FB	6	1.060556
13	DV	6	0.004796
14	FB	7	0.959605
15	DV	7	0.005210
16	FB	8	1.027295
17	DV	8	0.004640
18	FB	9	1.048851
19	DV	9	0.005034

Se ha guardado correctamente

▼ Visualización de resultados

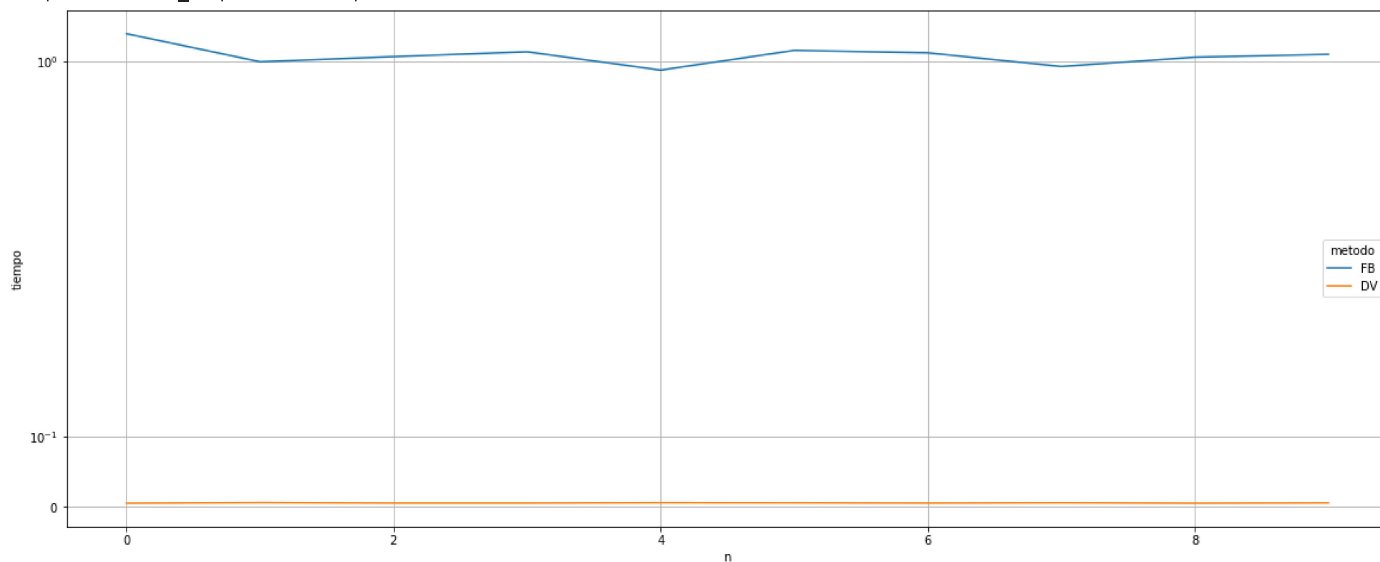
```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
f, ax = plt.subplots(figsize=(20, 8))
plt.xscale('linear')
plt.yscale('log')
ax.set_yscale('symlog', linthreshy=0.5)
```

```
ax.grid()
```

```
sns.lineplot(data = resultados,
             x = 'n',
             y = 'tiempo',
             hue='metodo',
             markers=True,
             dashes=False,
             ax=ax)
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f9cd0cc5910>



```
f, ax = plt.subplots(figsize=(20, 8))
plt.xscale('log')
plt.yscale('linear')
ax.set_yscale('symlog', linthreshy=0.5)
```

```
ax.grid()
```

Se ha guardado correctamente



po', y='metodo')

<matplotlib.axes._subplots.AxesSubplot at 0x7f9cd0c32a30>



Conclusión: Como podemos ver el el tiempo de ejecicón de los números más cercanos es considerablemente menos en el método de divide y vencerás, en el cual se comienza por ordenar la lista y luego hacer la comparación entre los elementos más cercanos. El tiempo promedio para divide y vencerás está en torno a los 4 milisegundos y el tiempo para la solución de fuerza bruta está al rededor de un segundo.



▼ Distancias para 2D

DV |

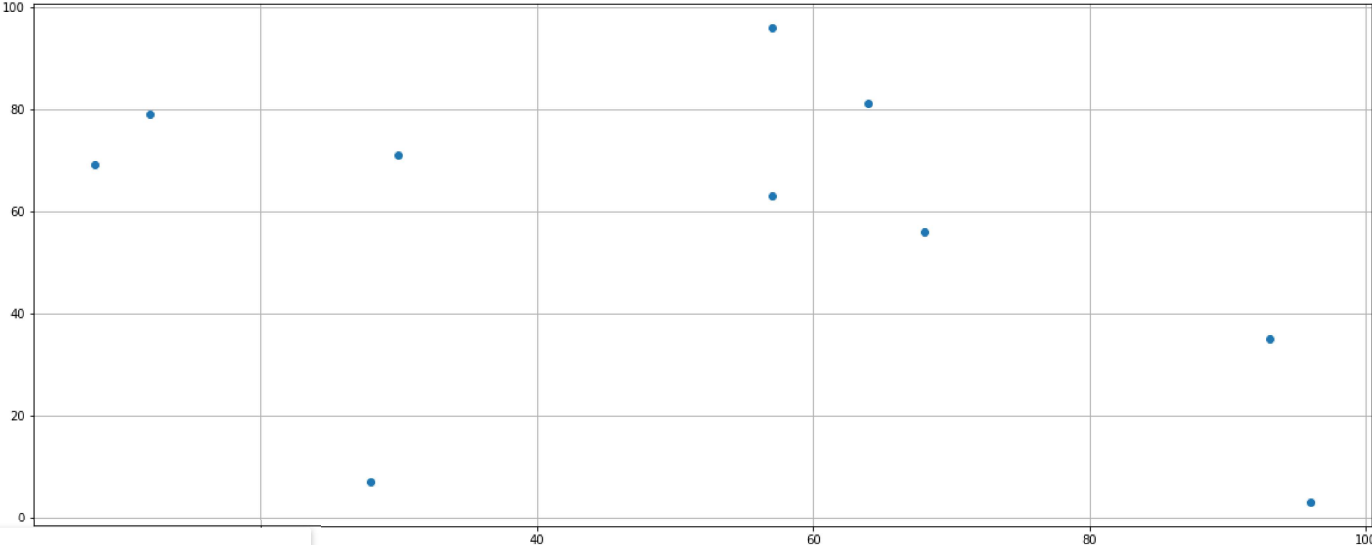
Se establecen parejas de datos la fila 0 contiene las coordenadas en X y la fila 1 y contiene las coordenadas en Y

```
lista_2D = np.random.randint(0,100, size = (2,10))
display(lista_2D)

array([[12, 64, 68, 57, 28, 93,  8, 30, 57, 96],
       [79, 81, 56, 63,  7, 35, 69, 71, 96,  3]])
```

```
plt.gcf().set_size_inches(20, 8)
plt.grid()
plt.scatter(lista_2D[0,:], lista_2D[1,:])
```

<matplotlib.collections.PathCollection at 0x7f95cab34eb0>



Se ha guardado correctamente

X

▼ Prueba

```

v1, v2 = lista_2D[0], 31
cercanos = minDistanciaFB_2D(lista_2D)

print(f'los puntos mas cercanos son: {cercanos[1][:]} y {cercanos[2][:]} con una distancia de: {cercanos[0]}')

```

```

los puntos mas cercanos son: (12, 79) y (8, 69) con una distancia de: 10.770329614269007
return (min(distancias, key=lambda x: x[0]))

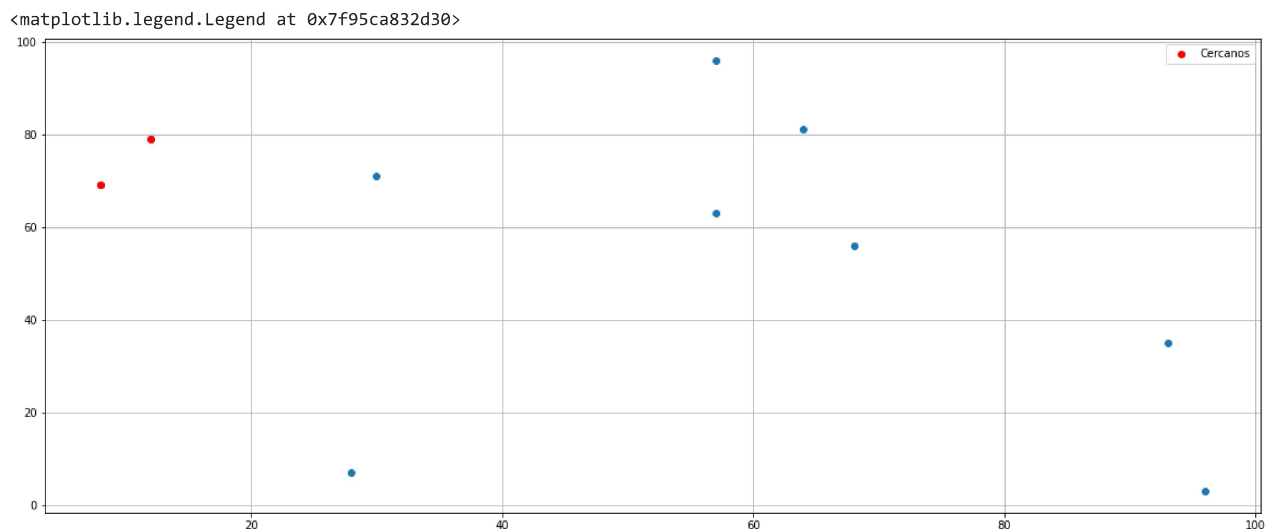
```

▼ Visualización

```

plt.gcf().set_size_inches(20, 8)
plt.grid()
plt.scatter(lista_2D[0,:], lista_2D[1,:])
plt.scatter(cercanos[1][0], cercanos[1][1], color='red', label='Cercanos')
plt.scatter(cercanos[2][0], cercanos[2][1], color='red')
plt.legend(loc='best')

```



Se ha guardado correctamente



[Productos de pago de Colab](#) - [Cancelar contratos](#)

✓ 0 s completado a las 9:31



No se ha podido establecer conexión con el servicio reCAPTCHA. Comprueba tu conexión a Internet y vuelve a cargar la página para ver otro reCAPTCHA.